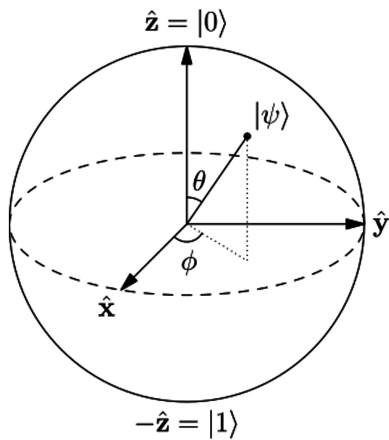


Quantum Coding with Qiskit

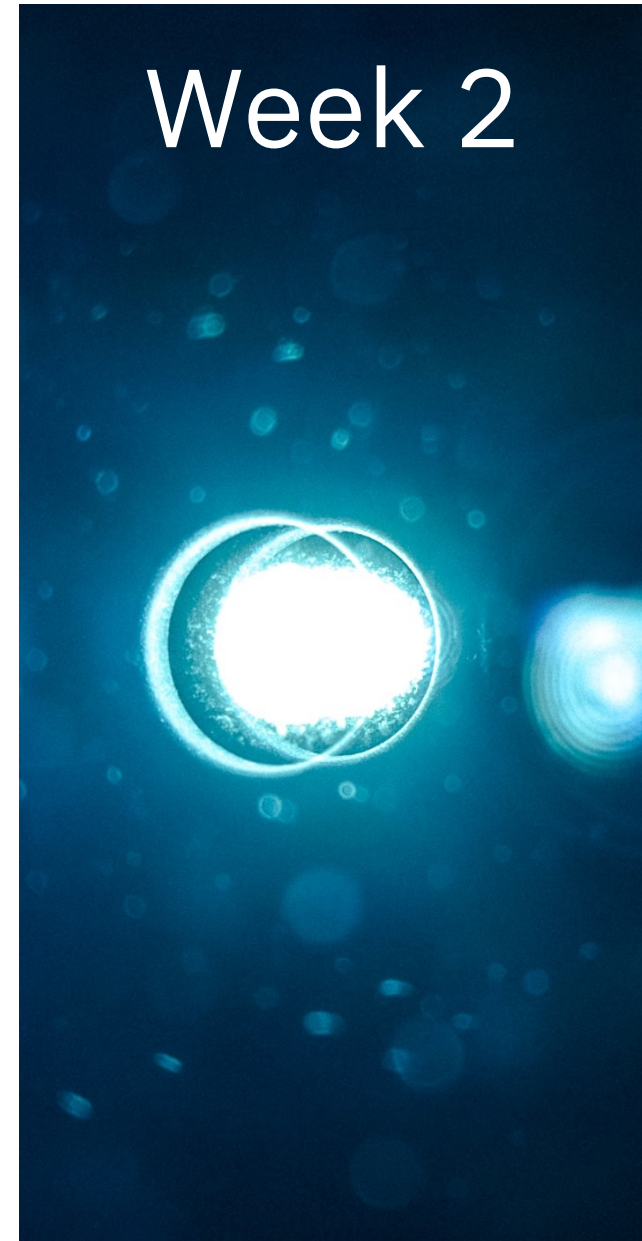
– A Course for Beginners

20th May 2026

Sebastian E. Grodzietzki, Dipl.-Inf.
Joey Rogers, M.Sc. Physics



Week 2



Welcome & Agenda / Week 2

Week 2 — Core Quantum Concepts / This is a **content week**

- Quantum Mechanics for Coders
 - ...in a Nutshell
 - 2-Level Quantum system
 - Quantum phase in detail
 - Measurement bases
- Coding: -> Notebook 1

You will learn how to visualize a vector (qubit) using Qiskit on the bloch sphere and how to rotate them: Single qubit gates, evolving a state with a circuit, measurement (simulator vs. real hardware)

 - Concepts:
 - Single qubits, state initialisation, manipulation and visualisation
 - Bloch sphere intuition
 - Rotation
 - Simulators
 - Measurement
- Q&A
- Optional Homework:

Coin Flip from Notebook 1 / intro to superposition, how to measure in a different axis

-> Great chance to interact and/or complete with your study group

Quantum Mechanics for Coders | In a Nutshell...

Uniquely quantum

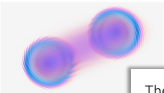
Some problems are classically intractable and will never be solvable with traditional computers



Superposition
A quantum system existing in a complex linear combination of two states until it is measured



Entanglement
Information shared jointly between entangled pairs or groups



Interference
Interaction that affects likelihood of solutions

These Quantum concepts can reduce the number of computational steps required for certain algorithms & At a ~100 qubit scale, with sufficient circuit depth and complexity, classical computers can no longer simulate exactly

© 2026 IBM Corporation

The practical path to useful quantum computing

Run quantum circuits **accurately** on quantum hardware

Map **interesting** problems to quantum circuits

100+ qubit systems and software

Methods that provide execution of quantum circuits on hardware with higher accuracy

Computational tasks that use those circuits

Industry and science applications that use those tasks

© 2026 IBM Corporation

Boeing

Exploring surface interactions in harsh environments

Hamiltonian simulation

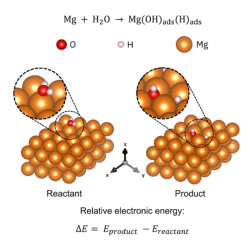
Boeing partnered with IBM to develop solutions for material surface reactions simulations relevant in the aerospace industry, focusing first on corrosion in magnesium, with the aim of improving material design and eventually product performance.

“It shows us that it’s not *if* quantum computers will be relevant to our business problems, but *when*.”

Jay Lowell
Chief Scientist, Disruptive Computing and Networks, Boeing

Running experiments of up to 20 qubits, the team demonstrated a novel methodology for simulating active spaces in surface material reactions, with application to water reduction in corrosion embedding method development could be used to study of complex periodic systems

© 2026 IBM Corporation



$Mg + H_2O \rightarrow Mg(OH)_{ads}(H)_{ads}$

Reactant Product

Relative electronic energy:
 $\Delta E = E_{product} - E_{reactant}$

Producing lower-cost, more energy-efficient fertilizer

Hamiltonian simulation

The production of nitrogen fertilizer currently takes ~2% of the world's energy and is responsible for between 1.4% and 5% of global greenhouse gas emissions.¹

Using sample-based quantum diagonalization (SQD) methods, IBM and RIKEN have simulated ground states of iron sulfur clusters [2Fe-2S] and [4Fe-4S] that exceed sizes amenable to exact diagonalizations, using up to 77 qubits of the IBM Quantum Heron processor.

Read the paper
[arXiv:2405.05068](https://arxiv.org/abs/2405.05068)

As Fe-S is the active center for the nitrogenase enzyme and iron-based catalysts for the Haber-Bosch process, SQD methods could help simulate FeMoco with increased precision, potentially yielding insights that could enhance food security, counter runoff, and increase crop revenue.

Source 1: “Nitrogen fertilizer production outstrips global needs and exceeds planetary boundaries by factor of two,” Planet Tracker, 14 Nov 2023.

© 2026 IBM Corporation

01. α^+ **Map** problem instance to quantum circuits and operators

Classical compute

02. $\bowtie$ **Optimize** for target hardware execution

Classical/Quantum Compute

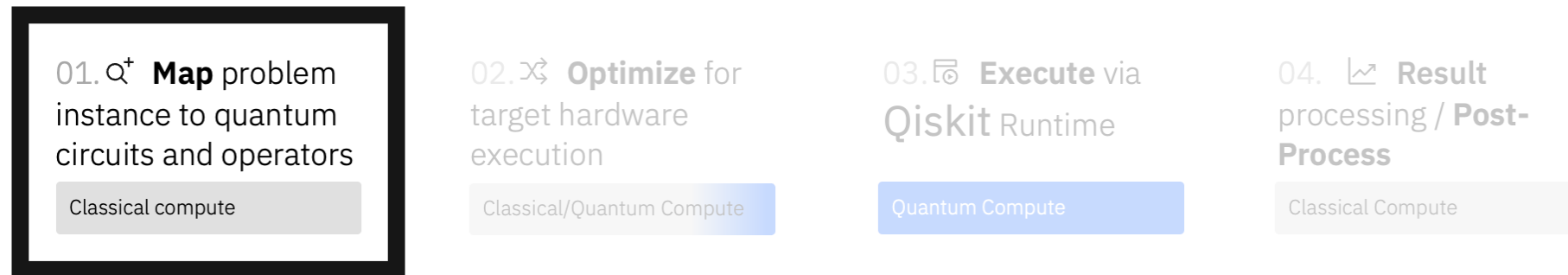
03. ⌚ **Execute** via Qiskit Runtime

Quantum Compute

04. ⬆ **Result** processing / **Post-Process**

Classical Compute

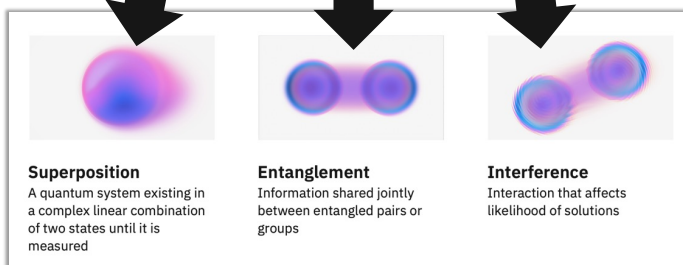
Quantum Mechanics for Coders | In a Nutshell...



Mapping:

We will use Qiskit to instantiate the number of Qubits we need, then you can put into **superposition**, **entangle** them (where we would like to correlate values), and change their phase to let (constructive or destructive) **interference** take place during runtime.

For all of this: we will use so-called gates that basically rotate the qubit (= the *state vector* representing it).



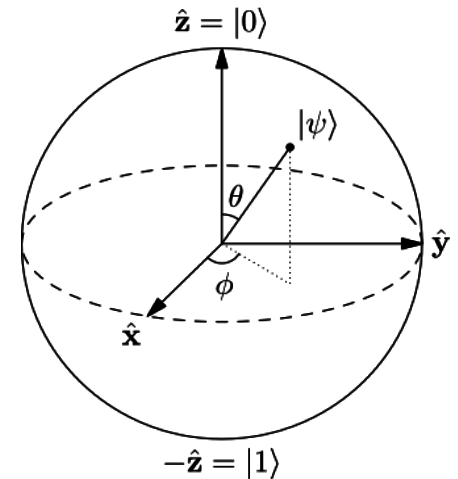
Quantum Mechanics for Coders | In a Nutshell...

The **Bloch sphere** is a geometric representation used to **visualize the state of a qubit** on a unit sphere. It illustrates pure states on the surface.

Furthermore, **quantum gates are visualized as rotations** on the sphere.

On the Bloch sphere, the poles represent the classical states:

- **North Pole:** The state $|0\rangle$
- **South Pole:** The state $|1\rangle$
- **Equator:** Here lie the superpositions (mixtures of 0 and 1).
- **The vector:** An arrow from the center to the surface indicates the current state of the qubit.



Quantum gates: In the quantum world, gates are essentially rotations of this arrow around specific axes. E.g.:

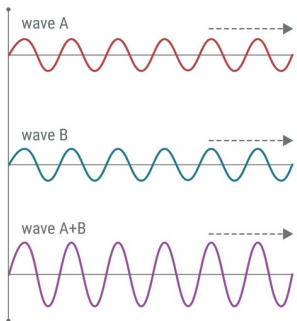
- **X-gate (the NOT-gate):** It rotates the arrow 180° around the X-axis. If you start at the North Pole, you end up at the South Pole.
- **Hadamard Gate (H-Gate):** Arguably the most important gate. It is a combination of rotations that shifts the arrow from the North Pole to the equator. This **moves the qubit from a fixed state into a superposition**.

Quantum Mechanics for Coders | In a Nutshell...

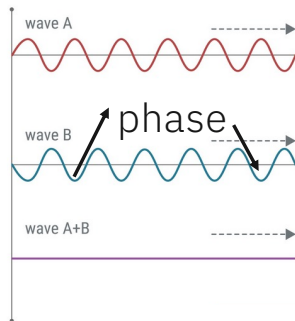
So, to **code** with Qubits, we basically rotate them across the X, Y, and Z-axis. The rotation **angles** (0° to 180°) are used to map/store data into the quantum circuit. To let **interference** happen between Qubits we rotate them to change/use their phase (= the angle/direction of the wave, the probability amplitudes).

Wave Interference

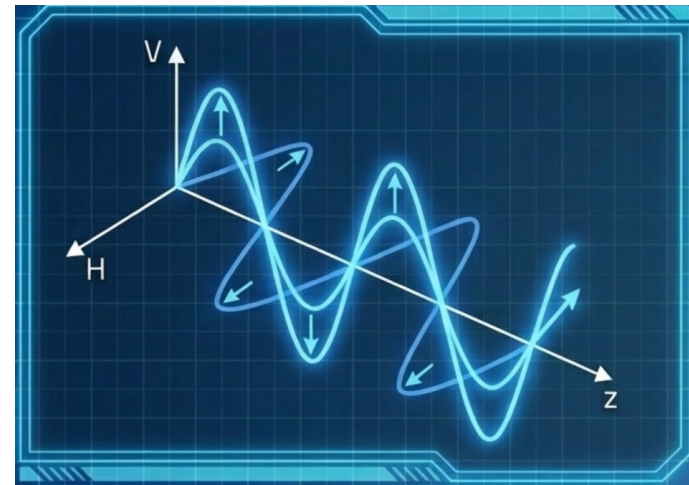
constructive interference



destructive interference



Now moving to
more dimensions
within Quantum



Source: NotebookLM from "Joseph M. Renes Quantum Information Theory"

Quantum Mechanics for Coders | In a Nutshell...

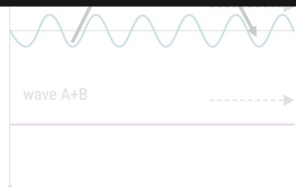
So, to code with Qubits, we basically rotate them across the X, Y, and Z-axis.

The rotation angles (0° to 180°) are used to map/store data into the quantum circuit.

To be
ang



Basically like having a radio with 3 buttons for 3 dimensions (X, Y, Z) to change the (electro magnetic) wave (= vector,) of our qubits.



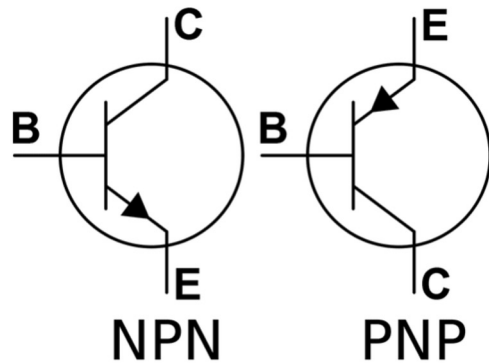
within Quantum



Source: NotebookLM from "Joseph M. Renes Quantum Information Theory"

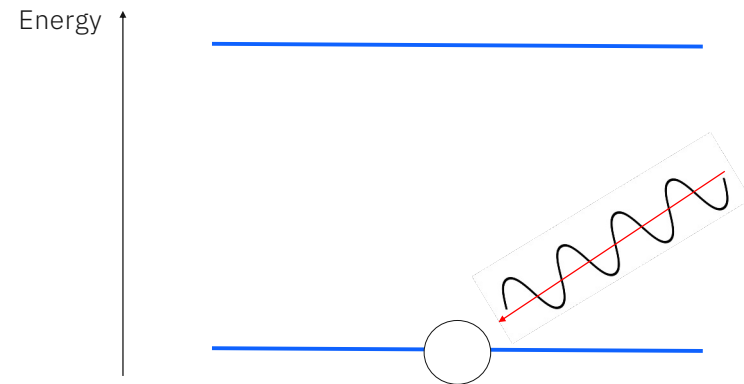
2-Level Quantum system

Classical System



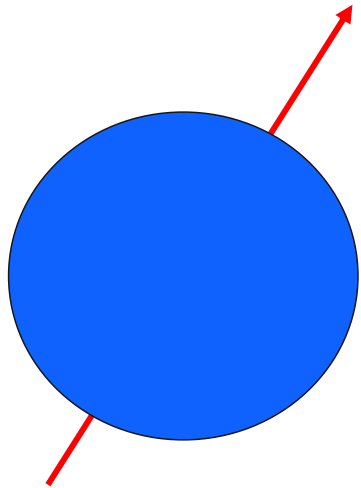
Apply a voltage, electrons flow

Quantum System



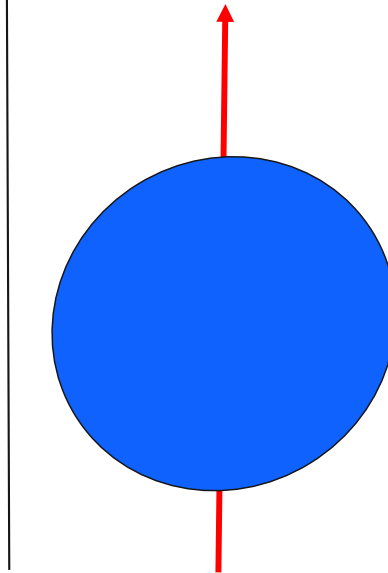
Isolate system to two addressable energy levels. Use pulses to control which energy level the qubit is in.

Spin Qubit System



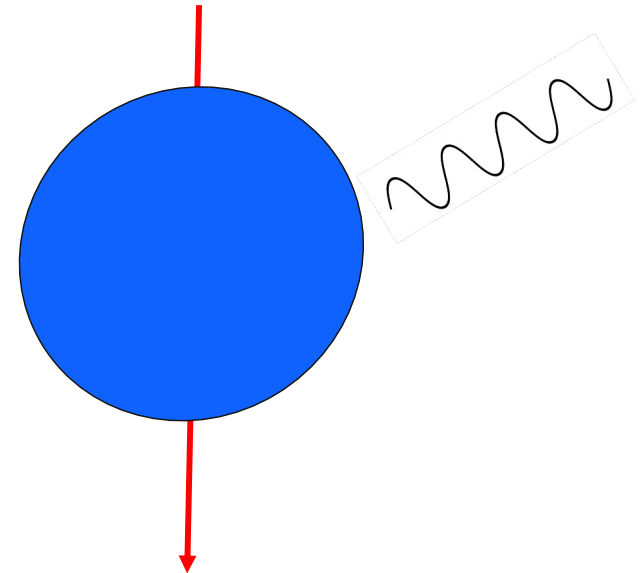
Free electron, random spin orientation.

Applied Magnetic Field



Electron in magnetic field, spin aligns with field.

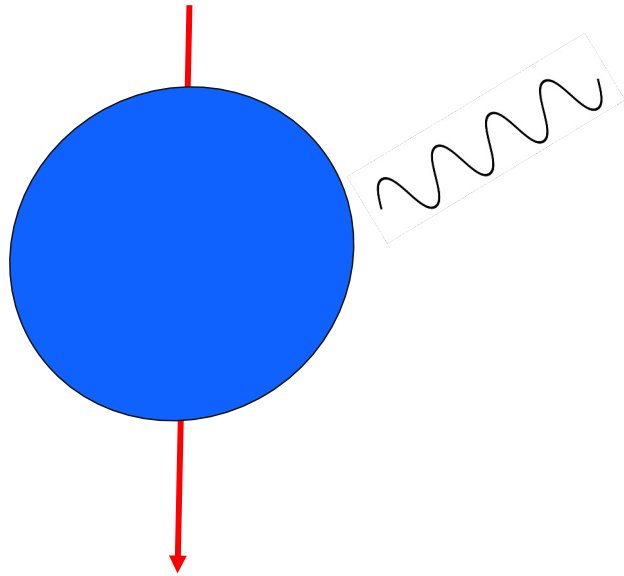
Applied Magnetic Field



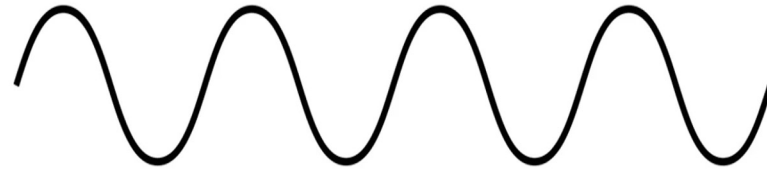
Apply (very specific) pulse, for a (very specific) amount of time to flip the spin.

Spin Qubit System

Applied Magnetic
Field



Apply (very specific) pulse,
for a (very specific)
amount of time to flip the
spin.

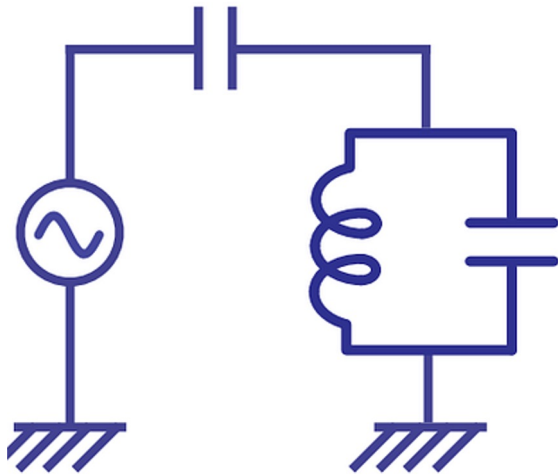


The details of this pulse are how we precisely control the qubit.

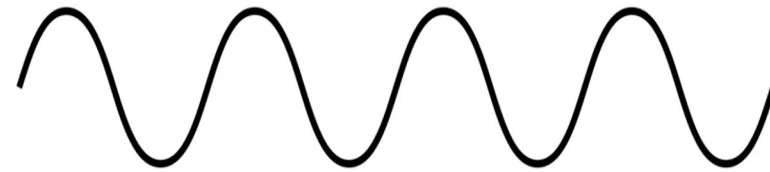
If you apply a pulse of exactly the right frequency for exactly the right amount of time, with exactly the right phase you can flip the qubit.

If you change some of these details, you can apply other quantum operations.

Superconducting Qubit System



In a SC system, your two levels are quantized amplitudes of electric current oscillating through a Jopheson junction.



The details of this pulse are how we precisely control the qubit.

If you apply a pulse of exactly the right frequency for exactly the right amount of time, with exactly the right phase you can flip the qubit.

If you change some of these details, you can apply other quantum operations.

Coding | Notebook 1

1-single-qubit-gates-and-visualisation.ipynb:

You will learn how to visualize a vector (qubit) on the bloch sphere and how to rotate them: Single qubit gates, evolving a state with a circuit, measurement (simulator vs. real hardware)

Concepts:

- Single qubits, state initialisation, manipulation and visualisation
- Bloch sphere intuition
- Rotation
- Simulators
- Measurement

Optional Homework: Coin Flip / intro to superposition, how to measure in a different axis
-> Great chance to interact and/or complete with your study group

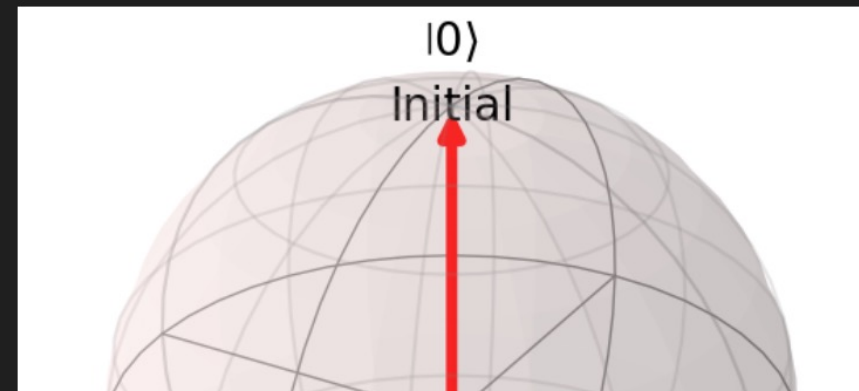
```
# 2. Build circuit
qc: QuantumCircuit = QuantumCircuit(1)
qc.h(qubit=0)

# 3. Evolve state
final_state: Statevector = initial_state.evolve(other=qc)

# 4. Print statevectors
print("Initial statevector:", initial_state.data)
print("Final statevector:", final_state.data)

# 5. Plot Bloch vectors
plot_bloch_states(
    vectors=[
        statevector_to_bloch(statevec=initial_state.data),
        statevector_to_bloch(statevec=final_state.data)
    ],
    labels=["Initial", "Final"],
    colors=["r", "b"]
)
```

```
Initial statevector: [1.+0.j 0.+0.j]
Final statevector: [0.70710678+0.j 0.70710678+0.j]
```



Q & A

Thank you!!

See you next Wednesday,
same place, 13:00-14:30

Sebastian E. Grodzietzki, Dipl.-Inf.
Joey Rogers, M.Sc. Physics

